

Building AI Agents

Architecture, Reliability, and Production Operations

Diogo Ribeiro

ESMAD - Escola Superior de Média Arte e Design

Lead Data Scientist, Mysense.ai

✉ dfr@esmad.ipp.pt ⓘ 0009-0001-2022-7072

May 24, 2026

Outline

- 1 Session Plan
- 2 Agent Fundamentals
- 3 Architecture and Orchestration
- 4 Tools, State, and Memory
- 5 Safety, Evaluation, and Operations
- 6 Case Study Walkthrough
- 7 Closing

Session Plan

90-Minute Session Plan

Time	Block
0-10 min	Agent definitions and decision criteria
10-30 min	Architecture and orchestration patterns
30-50 min	Tool contracts, memory, retrieval
50-70 min	Safety, evaluation, and observability
70-85 min	End-to-end case study walkthrough
85-90 min	Final checklist and Q&A

Delivery Modes

Use this same deck in three modes:

- **Core 60 min:** skip slides marked as “Deep Dive”
- **Standard 75 min:** include selected Deep Dive slides
- **Full 90 min:** run all slides with checkpoints and discussion

Deep Dive marker

Slides labeled **Deep Dive** are optional expansion content.

Learning Outcomes

By the end of this session, you should be able to:

- Decide when to use an agent vs a deterministic workflow
- Design an agent architecture with explicit control boundaries
- Define robust tool contracts and memory policies
- Build an evaluation loop that measures quality and cost
- Identify failure modes before they reach production

Agent Fundamentals

What Is an AI Agent?

Production Agent

A software system that iteratively selects actions, uses tools, updates state, and optimizes for a task objective under safety and cost constraints.

Properties that matter:

- Explicit objective and termination criteria
- Observable state transitions
- Typed integration surfaces
- Safety and governance controls

Agent vs Single LLM Call

Single LLM call

- One-shot response
- No execution state
- Limited external action
- Best for simple transformations

Agentic system

- Iterative control loop
- Plan, act, observe, adapt
- Tool invocation and validation
- Best for multi-step workflows

When You Should Not Use an Agent

Common overengineering trap

If a deterministic pipeline solves the problem with lower cost and higher reliability, do not use an agent.

- Static ETL transformations
- Low-latency hard real-time tasks
- Strictly regulated deterministic outputs
- Fixed, non-branching logic

Decision Matrix: Agent or Workflow?

Criterion	High	Low
Task ambiguity	Agent	Workflow
Branching depth	Agent	Workflow
Need for autonomy	Agent	Workflow
Regulatory strictness	Workflow	Agent
Acceptable latency	Workflow	Agent

Transition: From Definitions to Design

Next

We now move from **what an agent is** to **how to structure one** so behavior is predictable and auditable.

Architecture and Orchestration

Reference Architecture Layers

- **Interaction:** request intake, UX, response contracts
- **Policy:** authorization, data governance, action gating
- **Orchestration:** planner, executor, retry, stop logic
- **Model:** reasoning, extraction, ranking, verification
- **Tools:** APIs, DB, workflows, code execution wrappers
- **State:** working memory, session log, long-term knowledge
- **Observability:** traces, metrics, replay artifacts

Control Loop as a Deterministic Core

$$s_{t+1} = T(s_t, a_t, o_t), \quad a_t = \pi_{\theta}(s_t)$$

- s_t : current state (goal, plan, context, constraints)
- a_t : proposed action (tool call, clarification, finalize)
- o_t : observed output (tool result, validation status)
- T : deterministic transition logic in application code

Engineering rule

Keep control transitions deterministic. Isolate model stochasticity to action proposal.

Execution Patterns and Trade-offs

Pattern	Strength	Risk	Use Case
Router	Cheap, simple	Limited depth	Support triage
ReAct	Adaptive	Drift loops	Research/operations
Plan/Execute	Auditable	Planning overhead	Long workflows
DAG workflow	Predictable	Less adaptive	Enterprise flows
Multi-agent	Specialization	Coordination cost	Complex domains

A production agent must know when to stop.

- Goal achieved with validated outputs
- Budget exhausted (tokens, latency, tool-call quota)
- Confidence below threshold with no safe next action
- Policy gate requires human decision
- Repeated failure class without recovery path

Checkpoint 1: Architecture Review

Quick review prompt

For your current use case, define:

- Agent objective
- Required tools
- Termination conditions
- Mandatory human approval points

Tools, State, and Memory

Tool Contract Principles

- Strong input schema
- Deterministic output envelope
- Explicit error taxonomy
- Idempotency key for mutating calls

```
1 {  
2   "ok": true,  
3   "data": {...},  
4   "error_code": null,  
5   "trace_id": "abc-123"  
6 }  
7
```

Listing 1: Minimal tool envelope

Example: Mutation Tool Contract

```
1 {  
2   "name": "create_refund",  
3   "input_schema": {  
4     "type": "object",  
5     "properties": {  
6       "order_id": {"type": "string"},  
7       "reason_code": {"type": "string", "enum": ["damaged", "late",  
8         "wrong_item"]},  
9       "amount_cents": {"type": "integer", "minimum": 1}  
10    },  
11    "required": ["order_id", "reason_code", "amount_cents"]  
12  },  
13  "idempotency_key_required": true  
14 }
```

Listing 2: Refund action contract

State object should be explicit and minimal.

- Request metadata and user intent
- Active plan and completed steps
- Evidence references (not full raw payloads)
- Tool execution history with outcomes
- Policy flags and pending approvals

Memory Strategy

Working memory

- Active context
- Current constraints
- Latest tool outputs

Session memory

- Prior decisions
- User preferences
- Interaction history

Long-term memory

- Stable domain facts
- Playbooks
- Policy documents

Never persist

- Secrets
- Credentials
- Sensitive raw payloads

Retrieval and Grounding

- Retrieval is evidence support, not authority transfer
- Rank by relevance, trust, and freshness
- Attach provenance to every grounded claim
- Require citation for high-stakes outputs

$$U(d) = \alpha R(d) + \beta T(d) - \gamma S(d)$$

Checkpoint 2: Tool and Memory Design

Design exercise

Pick one high-risk tool in your domain and define:

- Input schema and output envelope
- Idempotency strategy
- Required approvals
- Memory retention policy

Safety, Evaluation, and Operations

Failure Taxonomy

- **Reasoning failures:** invalid decomposition, wrong assumptions
- **Tool failures:** timeouts, schema mismatch, side-effect duplication
- **Grounding failures:** hallucinated or stale claims
- **Policy failures:** unauthorized or unsafe action paths
- **Coordination failures:** state divergence across branches

Safety Controls That Actually Work

- Pre-action policy checks (role, scope, data class)
- Execution sandboxing and network egress controls
- Human approval gates for high-impact mutations
- Post-action reconciliation checks
- Immutable audit logs with trace ids

Three evaluation levels:

1. **Unit-level:** tool contract correctness
2. **Workflow-level:** scenario replay and regression sets
3. **Production-level:** online metrics and drift monitoring

Release gate

Ship only if all three levels pass quality and safety thresholds.

Quality

- Task success rate
- Plan validity
- Tool-call precision/recall
- Hallucination incidence

Operations

- p50/p95 latency
- Cost per successful task
- Retry rate per step
- Escalation rate

Observability and Debugging

- Per-step trace with prompt/tool/result metadata
- Correlation id across model and tool calls
- Structured error events with failure class
- Replay mode for deterministic postmortems
- Diff between expected and observed transitions

- Per-request token budgets
- Per-agent step limits
- Caching for repeated retrieval and deterministic calls
- Model routing: expensive models only for critical steps
- Cost-per-outcome monitoring, not just token tracking

Transition: From Components to End-to-End Delivery

Next

We now combine architecture, tooling, memory, and controls in one production case study.

Checkpoint 3: Reliability Readiness

Readiness questions

- Can you replay any failure from logs?
- Do you have a regression set for top workflows?
- Are mutation tools idempotent and approval-gated?
- Do you track cost per successful outcome?

Case Study Walkthrough

Case Study: Customer Support Resolution Agent

Goal: Resolve tickets with minimal human intervention while protecting policy and customer trust.

Tools:

- CRM lookup
- Order and shipment systems
- Refund/credit mutation endpoint
- Knowledge base retrieval

Case Study Flow

1. Classify issue and confidence level
2. Retrieve policy and customer/order evidence
3. Propose resolution plan with mutation requirements
4. Run policy checks and approval gate if needed
5. Execute mutation and verify post-state
6. Respond with explanation and audit metadata

Case Study: Plan Record

```
1 {  
2   "goal": "Resolve ticket #T-90821",  
3   "steps": [  
4     {"id": 1, "action": "lookup_order", "status": "done"},  
5     {"id": 2, "action": "check_policy", "status": "done"},  
6     {"id": 3, "action": "propose_refund", "status": "done"},  
7     {"id": 4, "action": "execute_refund", "status": "  
8       pending_approval"}  
9   ],  
10  "stop_if": ["approval_rejected", "budget_exceeded", "  
11    policy_violation"]  
}
```

Listing 3: Plan object persisted in state

Case Study: Failure and Recovery

Failure: refund API timeout after mutation accepted upstream.

Recovery strategy:

- Retry with same idempotency key
- Reconcile state by querying source-of-truth ledger
- If inconsistent, escalate with trace id and pause automation
- Emit incident signal for reliability dashboard

From Pilot to Production

1. Start with read-only paths
2. Introduce one low-risk mutation tool
3. Add approval gate and post-action verification
4. Expand coverage by failure-class maturity
5. Scale only after metric stability for 2-4 weeks

Closing

Anti-Patterns to Avoid

- Prompt tuning before system boundary definition
- Untyped tool calls and implicit side effects
- No deterministic replay path
- Missing approval gates on mutating actions
- Shipping with no regression benchmark

Production Readiness Checklist

Before you deploy

- Objective and stop criteria are explicit
- Tool interfaces are typed and idempotent
- Transition logic is deterministic and auditable
- Safety controls exist pre-action and post-action
- Evaluation covers quality, reliability, latency, and cost
- Incident replay is operationally usable

Facilitator Timing Guide

- **By slide 10:** finish fundamentals
- **By slide 20:** finish architecture and orchestration
- **By slide 30:** finish tools, memory, retrieval
- **By slide 37:** finish safety/evaluation/operations
- **Slides 38+:** case study, summary, Q&A

Discussion prompts

- Which workflow in your team is best for first deployment?
- What is your highest-risk mutation action?
- Which metric will define success in the first 30 days?

Appendix: Deep Dives

Deep Dive: Planner Design Choices

- **Single-shot planner:** fast, but brittle under changing evidence
- **Rolling planner:** re-plans each step, more robust but costlier
- **Hybrid:** coarse global plan + local rolling re-plan

Recommendation

Start with hybrid planning for long-running workflows with external dependencies.

Deep Dive: Tool Error Taxonomy

Error Class	Retry?	Typical Handling
ValidationError	No	fix input / ask clarification
AuthError	No	escalate / refresh auth context
TimeoutError	Yes	bounded retries + backoff
ConflictError	Maybe	reconcile state, retry if safe
PolicyDenied	No	halt and request approval

Deep Dive: Retrieval Evaluation

- Build query set from real production intents
- Measure top-k hit rate and citation correctness
- Include freshness tests for time-sensitive policies
- Track failure by source tier (internal docs vs external content)

Deep Dive: Regression Harness

- Store fixed input tasks with expected outcomes
- Replay with deterministic tool mocks when possible
- Compare plan traces, not only final answers
- Fail release on safety regressions even if accuracy improves

Deep Dive: Rollout Strategy

1. Internal alpha with shadow mode
2. Limited beta with read-only or low-risk actions
3. Gradual percentage rollout with automatic rollback trigger
4. Full rollout after stable SLO and incident trend